

Rapport de Projet IA01 : Prédiction de Problèmes de Santé

Mohamed Amine Saidane

22 octobre 2025

Table des matières

1	Introduction	2
2	Prétraitement des Données (X_train)	2
2.1	Chargement et Imputation des Données	2
2.2	Encodage Numérique (Label Encoding)	3
2.3	Sélection des Caractéristiques (Feature Selection)	3
2.4	Finalisation de X et y	4
3	Sélection du Modèle	4
3.1	Modèle 1 : Arbre de Décision	4
3.2	Métrique d'Évaluation : F1-Score Macro	5
3.3	Choix du Modèle Final	7
4	Entraînement Final et Génération des Prédictions	7
4.1	Traitement de l'Ensemble de Test (X_test)	7
4.2	Entraînement et Prédiction Finale	7
4.3	Formatage de la Soumission	8
5	Conclusion	9

1 Introduction

L'objectif de ce projet est de développer un modèle d'apprentissage automatique capable de prédire la classe de `probleme_sante` (Aucun, Faible, Modere) d'un individu en se basant sur un ensemble de caractéristiques de santé et de style de vie.

Pour ce faire, nous avons implémenté un pipeline complet en Python, en nous basant sur les fonctions fournies par le cours (`ia01.utils`, `ia01.arbre`, etc.). Ce rapport détaille les étapes de prétraitement des données, la sélection du modèle par validation croisée, et enfin la génération des prédictions sur l'ensemble de test.

2 Prétraitement des Données (X_train)

Les données brutes (`X_train.csv`) contiennent des valeurs manquantes, des attributs non numériques, et nécessitent une transformation avant de pouvoir être utilisées par un algorithme d'arbre de décision.

2.1 Chargement et Imputation des Données

La première étape consiste à charger les données et à traiter les valeurs manquantes.

Raisonnement Les données brutes contiennent des valeurs manquantes (ex : `age="0"`, `pays=""`). Les ignorer réduirait la taille de notre jeu de données. Nous choisissons donc de les imputer (remplacer).

- Pour l'âge (`age`), nous utilisons la **médiane**, qui est une mesure robuste aux valeurs aberrantes, calculée sur l'ensemble d'entraînement.
- Pour le pays (`pays`), nous utilisons une valeur constante arbitraire, "Italie".
- Pour le sexe (`sexe`), nous alternons "Homme" et "Femme" pour les valeurs manquantes.

```
1 data = lecture_csv("data/X_train.csv")
2
3 age=[x["age"] for x in data]
4 median_age = quantile(age,0.5)
5
6 L=data.copy()
7 i=0
8 s=0
9 for d in L:
10     if data[i]["age"]=="0":
11         data[i]["age"] = median_age
12     if d["pays"] == "" :
13         data[i]["pays"]="Italie"
14     if d["sexe"]==" " :
15         if s==0:
16             data[i]["sexe"]="Homme"
17             s=1
18         else:
19             data[i]["sexe"]="Femme"
20             s=0
21     # ... (suite du traitement)
22     i+=1
```

Listing 1 – Imputation des données manquantes de X_train

2.2 Encodage Numérique (Label Encoding)

L'arbre de décision ne peut traiter que des valeurs numériques. Nous convertissons donc les attributs textuels binaires ou ordinaux en entiers.

Raisonnement Des caractéristiques comme `sommeil_qualite` ont un ordre intrinsèque (Excellente > Bonne > ...). Un encodage numérique simple (Label Encoding) permet de préserver cette relation. Il en va de même pour les attributs binaires (sexe, cigarette, alcool).

```
1 # ... (dans la boucle de traitement de X_train)
2 if d["sommeil_qualite"] == 'Mauvaise':
3     data[i]["sommeil_qualite"]=0
4 if d["sommeil_qualite"] == "Passable":
5     data[i]["sommeil_qualite"]=1
6 if d["sommeil_qualite"] == "Bonne":
7     data[i]["sommeil_qualite"]=2
8 if d["sommeil_qualite"]=="Excellente":
9     data[i]["sommeil_qualite"]=3
10
11 if d["niveau_stress"] == 'Faible':
12     data[i]["niveau_stress"]=0
13 if d["niveau_stress"] == "Moyen":
14     data[i]["niveau_stress"]=1
15 if d["niveau_stress"] == "Haut":
16     data[i]["niveau_stress"]=2
17
18 if d["sexe"] == 'Homme':
19     data[i]["sexe"]=0
20 if d["sexe"] == "Femme":
21     data[i]["sexe"]=1
22 # ... (etc. pour cigarette et alcool)
```

Listing 2 – Encodage des caractéristiques ordinales et binaires

2.3 Sélection des Caractéristiques (Feature Selection)

Toutes les caractéristiques ne sont pas forcément utiles.

Raisonnement Pour ce modèle, nous avons fait le choix de ne pas inclure les caractéristiques catégorielles `pays` et `profession`, ni les caractéristiques jugées redondantes ou non informatives comme `cafe_verre` (corrélée à `cafeine_mg`) et `sommeil_duree` (corrélée à `sommeil_qualite`). Elles sont donc supprimées.

```
1 # ... (dans la boucle de traitement de X_train)
2 del(d["cafe_verre"])
3 del(d["profession"])
4 del(d["pays"])
5 del(d["sommeil_duree"])
6
7 # Conversion finale en float
8 for k in d.keys():
9     float(d[k])
10 i+=1
```

Listing 3 – Suppression des caractéristiques non utilisées

2.4 Finalisation de X et y

Raisonnement (Encodage de y_train) La variable cible y_train (contenue dans y_train.csv) présentait un fort déséquilibre de classes (un concept de votre cours "Données imparfaites"). L'ensemble des données contenait 4 classes : "Aucun", "Faible", "Modere", et "Severe".

Cependant, une analyse préliminaire a montré que la classe "Severe" était extrêmement rare, constituant **moins de 1%** de l'ensemble d'entraînement. Maintenir une classe aussi peu représentée aurait rendu l'apprentissage du modèle très difficile et l'évaluation (notamment le F1-Score) instable.

Pour créer un modèle plus robuste, nous avons pris la décision de **fusionner la classe "Severe" avec la classe "Modere"**. Notre problème de classification a donc été ramené à 3 classes :

- 0 : "Aucun"
- 1 : "Faible"
- 2 : "Modere" (regroupant "Modere" et "Severe")

Mise en œuvre Nous avons converti la liste de dictionnaires data en une liste de listes X et nous avons encodé la cible y_train en suivant cette nouvelle répartition à 3 classes.

```
1 y_train = lecture_csv("data/y_train.csv")
2
3 L=y_train.copy()
4 for i in range(len(L)):
5     if L[i]["probleme_sante"] == "Aucun":
6         y_train[i]=0
7     elif L[i]["probleme_sante"] == "Faible":
8         y_train[i]=1
9     else :
10        # Regroupe "Modere", "Severe"
11        y_train[i]=2
12
13 X=[]
14 for i in range(len(data)):
15     y=[]
16     for k in data[i].keys():
17         y.append(float(data[i][k]))
18     X.append(y)
```

Listing 4 – Encodage de la cible y_train (avec fusion de classes)

3 Sélection du Modèle

Pour déterminer le meilleur algorithme pour notre problème, nous avons comparé deux approches fondamentales de l'apprentissage supervisé : l'Arbre de Décision et les k-Plus Proches Voisins (k-PPV). L'objectif est de trouver le modèle et les hyperparamètres qui maximisent le F1-Score Macro lors d'une validation croisée.

3.1 Modèle 1 : Arbre de Décision

Stratégie de Validation Pour l'Arbre de Décision, l'hyperparamètre clé est sa profondeur maximale (max_prof). Pour éviter le sur-apprentissage et obtenir une estimation

fiable de la performance, nous utilisons une **validation croisée (K-Fold)** avec $K=5$. L'ensemble d'entraînement X est divisé en 5 blocs. Le modèle est entraîné 5 fois (sur 4 blocs) et évalué sur le bloc restant.

```

1 y=y_train.copy()
2 K = 5
3 X_K, y_K = partition_val_croisee(X, y, K)
4
5 prof = list(range(1, 12)) + [float("inf")] # Profondeurs a tester
6 f_cv = [0] * len(prof) # Initialisation du score F1
7
8 for i in range(K):
9     X_val, y_val = X_K[i], y_K[i]
10    X_train, y_train = [], []
11    for j in range(K):
12        if j != i:
13            X_train += X_K[j]
14            y_train += y_K[j]
15
16    # Entrainement sur le fold
17    arbre = arbre_train(X_train, y_train)
18
19    # Evaluation sur differentes profondeurs
20    for j, p in enumerate(prof):
21        y_pred_val = arbre_pred(X_val, arbre, max_prof=p)
22        # ... (calcul des metriques)

```

Listing 5 – Mise en place de la validation croisée ($K=5$)

3.2 Métrique d'Évaluation : F1-Score Macro

Raisonnement Le problème est une classification multi-classe (0, 1, 2) et potentiellement déséquilibrée. Le simple taux d'erreur est trompeur. Nous utilisons donc le **F1-Score Macro** : nous calculons le F1-score pour chaque classe (0, 1, et 2) et nous prenons la moyenne arithmétique. Cela donne une importance égale à chaque classe.

```

1 # ... (dans la boucle de validation)
2 for j, p in enumerate(prof):
3     y_pred_val = arbre_pred(X_val, arbre, max_prof=p)
4
5     # Calcul du F1-Score Macro (moyenne sur les classes 0, 1, et 2)
6     for j, p in enumerate(prof):
7         y_pred_val = arbre_pred(X_val, arbre, max_prof=p)
8         erreur_cv[j] += taux_erreur(y_val, y_pred_val) / K
9         prec_cv[j] += (precision(y_val, y_pred_val, 1)
10        + precision(y_val, y_pred_val, 2)
11        + precision(y_val, y_pred_val, 0)) / (3*K)
12        rap_cv[j] += (rappel(y_val, y_pred_val, 2)
13        + rappel(y_val, y_pred_val, 1)
14        + rappel(y_val, y_pred_val, 0)) / (3*K)
15        f_cv[j] += (f_score(y_val, y_pred_val, 0)
16        + f_score(y_val, y_pred_val, 2)
17        + f_score(y_val, y_pred_val, 1)) / (3*K)
18
19 # Trouver la profondeur qui maximise le F1-Score Macro
20 bestp_f = argsort(f_cv, True)[0]

```

```
21 print(f"Meilleur F1-score : prof = {prof[bestp_f]}")
```

Listing 6 – Calcul du F1-Score Macro dans la boucle de CV

La validation croisée a identifié que la profondeur `prof[bestp_f]` (par exemple, 3) maximisait le F1-Score.

Modèle 2 : Tentative avec k-Plus Proches Voisins (k-PPV)

Stratégie de Validation En parallèle, nous avons testé le modèle des k-Plus Proches Voisins (k-PPV). Cet algorithme est fondamentalement différent et repose sur des calculs de distance.

Normalisation des données Le k-PPV est très sensible aux échelles des différentes caractéristiques (un IMC de 25 n'a pas la même "importance" qu'un `sexe` de 1). Il est donc **impératif** de normaliser les données avant d'utiliser ce modèle. Nous avons appliqué une normalisation de type "echelle" sur `X`.

```
1 # X a ete cree precedemment
2 y=y_train.copy()
3
4 # Normalisation "echelle"
5 loc1, scale1 = norm_param(X, "echelle")
6 X_norm1 = normalisation(X, loc1, scale1)
```

Listing 7 – Normalisation des données pour k-PPV

Résultats de la Validation Croisée (k-PPV) Nous avons ensuite effectué une validation croisée ($K=5$) sur ces données normalisées pour tester différentes valeurs de `k`, en utilisant le taux d'erreur comme métrique.

```
1 K = 5
2 X_K, y_K = partition_val_croisee(X_norm1, y, K)
3
4 for k in [1, 3, 5, 7, 9, 11, 13, 15, 17, 19, 21, 23, 25]:
5     erreur_cv = 0
6     for i in range(K):
7         X_val = X_K[i]
8         y_val = y_K[i]
9         X_train, y_train = [], []
10        for j in range(K):
11            if j != i:
12                X_train += X_K[j]
13                y_train += y_K[j]
14        # Calcul de l'erreur pour k-PPV
15        erreur_cv += taux_erreur(y_val, kppv(X_val, X_train, y_train, k)
16    )
17    erreur_cv /= K
18    print(f"Taux d'erreur pour k={k}" , " VC = " , erreur_cv)
```

Listing 8 – Validation croisée ($K=7$) pour k-PPV

Les résultats ont montré un taux d'erreur moyen élevé, se stabilisant autour de 15%-16% pour les meilleures valeurs de `k`.

3.3 Choix du Modèle Final

En comparant les performances :

- **k-PPV (normalisé)** : Taux d'erreur élevé ($\approx 15\%$).
- **Arbre de Décision** : A montré un F1-Score Macro bien supérieur (impliquant une erreur plus faible) pour des profondeurs faibles (ex : 3 ou 4).

Nous avons donc **écarté le modèle k-PPV** et choisi de retenir l'**Arbre de Décision** comme notre modèle final, avec la profondeur optimale `prof[bestp_f]` (par exemple, 3) identifiée par la validation croisée basée sur le F1-Score.

4 Entraînement Final et Génération des Prédictions

4.1 Traitement de l'Ensemble de Test (`X_test`)

Raisonnement Il est crucial d'appliquer exactement le même pipeline de **prétraitement** à `X_test` qu'à `X_train`. Nous réutilisons les mêmes médianes (`median_age`) et les mêmes logiques d'encodage et de suppression de colonnes pour éviter toute fuite de données ou incohérence.

```
1 data = lecture_csv("data/X_test.csv")
2 L=[]
3 i=0
4 L=data.copy()
5 for d in L:
6     if data[i]["age"]=="0":
7         data[i]["age"] = median_age # Utilisation de la mediane de
X_train
8     # ... (toutes les autres etapes d'imputation)
9
10    # ... (toutes les etapes d'encodage label)
11
12    # ... (suppression des memes colonnes)
13    del(d["cafe_verre"])
14    del(d["profession"])
15    del(d["pays"])
16    del(d["sommeil_duree"])
17
18    for k in d.keys():
19        float(d[k])
20    i+=1
```

Listing 9 – Traitement de `X_test` (identique à `X_train`)

4.2 Entraînement et Prédiction Finale

Nous entraînons un nouvel arbre de décision sur l'intégralité de nos données `X` et `y`. Ensuite, nous utilisons ce modèle pour prédire les classes de `X_test`.

Raisonnement La validation croisée a permis de choisir la profondeur optimale. Pour la soumission finale, nous avons sélectionné une profondeur de **3**, qui s'est avérée être un bon compromis (identifié par `bestp_f`). Le modèle final est entraîné sur 100% des données d'entraînement et prédit sur l'ensemble de test traité.

```

1 # Entraînement final sur 100% de X et y
2 arbre_final = arbre_train(X,y)
3
4 # Creation de X_test
5 X_test=[]
6 for i in range(len(data)):
7     y=[]
8     for k in data[i].keys():
9         y.append(float(data[i][k]))
10        X_test.append(y)
11
12 # Prediction sur X_test avec la profondeur choisie
13 y_test_pred = arbre_pred(X_test,arbre_final,max_prof=3)

```

Listing 10 – Entraînement final et prédiction sur X_test

4.3 Formatage de la Soumission

Les prédictions numériques (0, 1, 2) sont converties en étiquettes textuelles ("Aucun", "Faible", "Modere") et sauvegardées dans le fichier de soumission.

```

1 y_test=[]
2 for p in y_test_pred:
3     if p==0:
4         y_test.append("Aucun")
5     elif p==1:
6         y_test.append("Faible")
7     else:
8         y_test.append("Modere")
9
10 ecriture_csv_projet(y_test)

```

Listing 11 – Formatage et écriture du fichier de soumission

5 Conclusion

Ce projet nous a permis de mettre en œuvre un pipeline complet d'apprentissage automatique, de la lecture des données brutes à la génération d'un fichier de prédiction, en utilisant uniquement les outils de base de Python et les modules fournis par le cours.

Le défi principal résidait dans le prétraitement des données (`X_train.csv`), qui étaient significativement imparfaites. Nous avons appliqué une méthodologie rigoureuse pour les nettoyer et les transformer :

1. **Imputation des données manquantes** : L'âge a été imputé en utilisant la médiane (robuste aux extrêmes), tandis que d'autres attributs comme le sexe ont été complétés pour ne perdre aucune donnée.
2. **Encodage des caractéristiques** : Les attributs ordinaux (comme `sommeil_qualite`) ont été convertis en entiers pour préserver leur ordre, et les binaires (comme `cigarette`) en 0/1.
3. du déséquilibre de classe : Une étape cruciale a été l'analyse de la cible `y_train`. La classe "Severe", représentant moins de 1% des données, a été fusionnée avec la classe "Modere". Cette décision a permis de réduire le problème à 3 classes, le rendant plus stable et améliorant la capacité du modèle à apprendre des classes minoritaires.
4. **Sélection de caractéristiques** : Dans cette itération, les attributs catégoriels complexes (`pays`, `profession`) ont été écartés pour simplifier le modèle initial.

La phase de sélection du modèle a été déterminante. Nous avons comparé deux approches :

- Le **k-PPV**, qui, même après une normalisation "échelle" des données, a montré des performances insuffisantes, avec un taux d'erreur élevé avoisinant les 15%.
- L'**Arbre de Décision**, qui s'est révélé bien plus performant sur ce jeu de données.

Pour optimiser cet arbre, nous avons utilisé une validation croisée (K-Fold) en testant plusieurs profondeurs. Le choix de la métrique a été essentiel : nous avons privilégié le **F1-Score Macro** au lieu du simple taux d'erreur. Cette métrique, en donnant un poids égal à chaque classe, est la seule à garantir que notre modèle est performant sur l'ensemble des catégories (y compris "Faible" et "Modere") et pas seulement sur la classe majoritaire "Aucun".

La validation croisée a identifié une profondeur optimale de **3**. Ce modèle final, représentant un bon équilibre entre simplicité (évitant le sur-apprentissage) et capacité de discrimination, a été entraîné sur l'intégralité des données `X` et `y` traitées.

Finalement, l'ensemble de test `X_test` a subi le **même pipeline de transformation** (en utilisant la médiane de `X_train`), garantissant l'absence de fuite de données. Les prédictions générées par notre arbre optimisé ont été formatées et sauvegardées dans le fichier `y_test.csv` pour la soumission.